

Cloud Architecture Deployment: A Portfolio Project Report By Vishrut Pandey (9225380)

GitHub Repository: <https://github.com/siliconsagenerd/iu-cloud-programming-portfolio>

1. Project Overview & Outcome

This report summarizes the successful design, implementation, and deployment of a scalable, low-latency, and highly available web architecture on Amazon Web Services (AWS). The project's primary goal was to demonstrate a practical understanding of core cloud computing principles by hosting a simple webpage on an infrastructure defined entirely by code. The final product achieves this through a robust, multi-tier architecture that was planned in the conception phase, built and debugged during the development phase, and resulted in a fully functional, globally accessible web application.

2. The Technical Journey: From Concept to Reality

The application was built on a best-practice cloud architecture as planned in the conception phase. The core components were chosen to meet specific requirements: **AWS CloudFront** was selected to provide a low-latency Content Delivery Network (CDN); an **Application Load Balancer (ALB)** was used to distribute traffic for high availability; and an **EC2 Auto Scaling Group** was implemented to ensure the backend could scale automatically.

The entire infrastructure was provisioned using **Terraform**, an Infrastructure as Code (IaC) tool. This approach allowed for a repeatable, version-controlled, and automated deployment process, which is a fundamental principle of modern cloud engineering. The project was managed in a modular fashion, with separate Terraform files for networking, security, compute, and CDN resources, enhancing readability and maintainability.

3. Project Retrospective (The "Making Of")

- **What Went Well:** The initial architecture design proved to be very effective. The combination of CloudFront, an ALB, and an Auto Scaling Group is a standard industry pattern that successfully met all project requirements. Using Terraform as an IaC tool was invaluable. It provided a clear way to define the infrastructure and made the process of destroying and re-deploying the entire stack during troubleshooting incredibly efficient.
- **Challenges & Unforeseen Pitfalls:** The most significant technical challenge was diagnosing and resolving the **502 Bad Gateway** error. This was not a simple configuration mistake but a deeper networking issue. The root cause was that the private EC2 instances

had no outbound path to the internet, which prevented the `user_data` script from downloading and installing the web server software.

- **The Solution:** The issue was resolved by correctly implementing a **NAT Gateway** in a public subnet and adding a route to it from the private subnets' route table. This provided the necessary internet egress for the instances to initialize correctly, allowing them to pass the load balancer's health checks. This real-world troubleshooting experience was a critical part of the project.

4. Feature Spotlight: The High-Availability and Scaling System

The feature I am most proud of is the fully functional, self-healing, and scalable backend system. This is more than just a collection of servers; it's a dynamic system that works together to ensure reliability. The integration of the **Application Load Balancer's health checks** with the **EC2 Auto Scaling Group** means the system can automatically detect and replace a faulty instance without any manual intervention. This demonstrates a core value proposition of the cloud: building resilient systems that can withstand failure and adapt to changing demand.

5. Future Plans

While the current project is a complete implementation of the initial task, several enhancements could be made in a real-world scenario:

- **CI/CD Pipeline:** The next logical step would be to implement a Continuous Integration/Continuous Deployment (CI/CD) pipeline (e.g., using GitHub Actions). This would automate the process of testing and deploying changes to the Terraform code, further improving reliability and development speed.
- **Advanced Monitoring and Logging:** Implementing a more robust monitoring solution like Amazon CloudWatch Dashboards and centralized logging would provide deeper insights into the application's performance and help diagnose issues more quickly.
- **Containerization:** The application could be containerized using Docker and deployed to a managed service like Amazon ECS or EKS. This would improve portability and streamline the deployment of application updates.

6. Conclusion

This project successfully progressed from a well-defined architectural concept to a fully realized, reliable, and globally accessible cloud application. The journey underscored the importance of a solid networking foundation, the power of Infrastructure as Code for automation, and the critical need for systematic troubleshooting when faced with complex, real-world challenges. The final product is a testament to these principles, delivering a high-quality cloud deployment that meets all technical requirements.